Presentación 1 IA

Nicolás Chehade Casivar

Temas

- Representación
- Representación implementada
- Restricciones Implementadas
- Función evaluación
- Función evaluación implementada
- Lectura
- Greedy propuesto
- Solución inicial con Greedy
- Movimiento propuesto
- Trabajo pendiente
- Conclusiones

Representación

‘Lista con listas de listas, con cada sublista de primer nivel representando una clase, con tres elementos dentro: <Dia, Período, Sala>.

C: C1 y C2
D: Lunes y martes
P: P1 y P2
R: R1
L: C1 requiere 2 clases y C2 requiere 1 clase. (3 ttal)

Una solución sería:

```
S = [
        [(Lunes, P1, R1), (Martes, P2, R1)],   //C1
        [(Martes, P1, R2)]                     //C2
        ]
```

Tamaño: 3*|L| (+ |C| si se consideran sublistas)
En particular: 9/11

Representación Implementada

```
43  struct Instancia {
44      string nombre;
45      int courses;
46      int rooms;
47      int days;
48      int periods_per_day;
49      int curricula;
50      int constraints;
51
52
53      vector<Curso> cursos;
54      vector<Sala> salas;
55      vector<Curricula> curriculas;
56      vector<Disponibilidad> restricciones;
57  };
58
```

Restricciones implementadas

```
1
2   #ifndef RESTRICCIONES_H
3   #define RESTRICCIONES_H
4
5   #include "estructuras.h"
6
7   // H1: Lectures
8   bool validarH1(const Instancia& instancia, const Solucion& solucion);
9
10  // H2: Room Occupancy
11  bool validarH2(const Instancia& instancia, const Solucion& solucion);
12
13  // H3: Conflict
14  bool validarH3(const Instancia& instancia, const Solucion& solucion);
15
16  // H4: Availability
17  bool validarH4(const Instancia& instancia, const Solucion& solucion);
18
19  //Validacion de todas
20  bool validarTodas(const Instancia& instancia, const Solucion& solucion);
21
22  #endif
23
```

```cpp
//H1
bool validarH1(const Instancia& instancia, const Solucion& solucion) {
    (void)instancia;
    for (const auto& asignaciones : solucion.cursos) {

        for (size_t i = 0; i < asignaciones.size(); i++) {
            for (size_t j = i + 1; j < asignaciones.size(); j++) {
                // Si dia, periodo y curso coinciden. False
                if (asignaciones[i].dia == asignaciones[j].dia &&
                    asignaciones[i].periodo == asignaciones[j].periodo) {
                    return false;
                }
            }
        }
    }

    return true;
}
```

```cpp
//H2
bool validarH2(const Instancia& instancia, const Solucion& solucion) {
    (void)instancia;
    for (size_t i = 0; i < solucion.cursos.size(); i++) {
        for (const auto& asignacion1 : solucion.cursos[i]) {

            for (size_t j = i + 1; j < solucion.cursos.size(); j++) {
                for (const auto& asignacion2 : solucion.cursos[j]) {
                    // Si dia, periodo y sala coindicen, False
                    if (asignacion1.dia == asignacion2.dia &&
                        asignacion1.periodo == asignacion2.periodo &&
                        asignacion1.sala == asignacion2.sala) {
                        return false;
                    }
                }
            }
        }
    }

    return true;
}
```

```cpp
//H3
bool validarConflictoProfesor(const Instancia& instancia, const Solucion& solucion) {

    for (size_t i = 0; i < solucion.cursos.size(); i++) {

        for (size_t j = i + 1; j < solucion.cursos.size(); j++) {

            // Si es diferente profesor, va al siguiente
            if (instancia.cursos[i].profesor != instancia.cursos[j].profesor) {
                continue;
            }

            // Aquí ya sé que curso i y curso j comparten profesor.
            for (const auto& a1 : solucion.cursos[i]) {
                for (const auto& a2 : solucion.cursos[j]) {
                    //Si tienen el mismo profesor, no pueden compartir día y período.
                    if (a1.dia == a2.dia &&
                        a1.periodo == a2.periodo) {
                        return false;
                    }
                }
            }
        }
    }

    return true;
}
```

```cpp
bool validarH3(const Instancia& instancia, const Solucion& solucion) {

    return validarConflictoProfesor(instancia, solucion) &&
           validarConflictoCurricula(instancia, solucion);
}
```

```cpp
bool validarConflictoCurricula(const Instancia& instancia, const Solucion& solucion) {

    for (size_t i = 0; i < solucion.cursos.size(); i++) {

        for (size_t j = i + 1; j < solucion.cursos.size(); j++) {

            bool compartenCurricula = false;

            // Buscar alguna curricula que contenga ambos cursos
            for (const auto& curricula : instancia.curriculas) {

                bool contieneI = false;
                bool contieneJ = false;

                for (const auto& curso : curricula.cursos) {

                    if (curso == instancia.cursos[i].id) {
                        contieneI = true;
                    }

                    if (curso == instancia.cursos[j].id) {
                        contieneJ = true;
                    }
                }

                if (contieneI && contieneJ) {
                    compartenCurricula = true;
                    break;
                }
            }
            //Si no comparten ninguna curricula, vamos a la siguiente
            if (!compartenCurricula) {
                continue;
            }

            // Aquí ya sé que comparten curricula curso i y curso j
            for (const auto& a1 : solucion.cursos[i]) {
                for (const auto& a2 : solucion.cursos[j]) {
                    // Dos mallas de la misma curricula no pueden compartir día y period (tope)
                    if (a1.dia == a2.dia &&
                        a1.periodo == a2.periodo) {
                        return false;
                    }
                }
            }
        }
    }

    return true;
}
```

```cpp
//H4
bool validarH4(const Instancia& instancia, const Solucion& solucion) {

    for (size_t i = 0; i < solucion.cursos.size(); i++) {

        const string& idCurso = instancia.cursos[i].id;

        for (const auto& asignacion : solucion.cursos[i]) {

            for (const auto& restriccion : instancia.restricciones) {
                //Dada un curso, revisa que cumpla con la restriccion de disponibilidad.
                if (restriccion.curso == idCurso &&
                    restriccion.dia == asignacion.dia &&
                    restriccion.periodo == asignacion.periodo) {

                    return false;
                }
            }
        }
    }

    return true;
}

```

Función de evaluación

Sería la misma función objetivo del modelo planteado:

f. eval: MIN Z = S1+5S2+2S3+S4

Donde:

- S1: Room Capacity
- S2: Minimum Working Days
- S3: Curriculum Compactness
- S4: Room Stability

Dado lo descrito en [1].

Función de evaluación implementada

Presentación 1 > codes > C f_eval.h

```h
#ifndef F_EVAL_H
#define F_EVAL_H

#include "estructuras.h"

int evaluarS1(const Instancia& instancia, const Solucion& solucion);
int evaluarS2(const Instancia& instancia, const Solucion& solucion);
int evaluarS3(const Instancia& instancia, const Solucion& solucion);
int evaluarS4(const Instancia& instancia, const Solucion& solucion);

int f_eval(const Instancia& instancia, const Solucion& solucion);

#endif
```

```cpp
int f_eval(const Instancia& instancia, const Solucion& solucion) {
    int S1 = evaluarS1(instancia, solucion);
    int S2 = evaluarS2(instancia, solucion);
    int S3 = evaluarS3(instancia, solucion);
    int S4 = evaluarS4(instancia, solucion);

    return S1 + 5 * S2 + 2 * S3 + S4;
}
```

```cpp
//S1: Room Capacity
int evaluarS1(const Instancia& instancia, const Solucion& solucion) {
    (void) instancia;
    int penalizacion = 0;

    for (size_t i = 0; i < solucion.cursos.size(); ++i) {
        const Curso& curso = instancia.cursos[i];

        for (const Asignacion& asignacion : solucion.cursos[i]) {
            for (const Sala& sala : instancia.salas) {
                if (sala.id == asignacion.sala) {
                    penalizacion += max(0, curso.students - sala.capacity);
                    break;
                }
            }
        }
    }

    return penalizacion;
}
```

```cpp
//S2: Minimum Working Days
int evaluarS2(const Instancia& instancia, const Solucion& solucion) {
    int penalizacion = 0;

    for (size_t i = 0; i < solucion.cursos.size(); ++i) {
        const Curso& curso = instancia.cursos[i];

        vector<bool> diasOcupados(instancia.days, false);

        for (const Asignacion& asignacion : solucion.cursos[i]) {
            diasOcupados[asignacion.dia] = true;
        }

        int diasUtilizados = 0;

        for (bool ocupado : diasOcupados) {
            if (ocupado) {
                ++diasUtilizados;
            }
        }

        penalizacion += max(0, curso.minWorkingDays - diasUtilizados);
    }

    return penalizacion;
}
```

```cpp
//S3: Curriculum Compactness (evitar ventanas)
int evaluarS3(const Instancia& instancia, const Solucion& solucion) {
    int penalizacion = 0;

    for (const Curricula& curr : instancia.curriculas) {

        vector<vector<bool>> ocupados(
            instancia.days,
            vector<bool>(instancia.periods_per_day, false)
        );

        // periodos de la curricula
        for (const string& idCurso : curr.cursos) {

            for (size_t i = 0; i < instancia.cursos.size(); ++i) {

                if (instancia.cursos[i].id == idCurso) {

                    for (const Asignacion& asignacion : solucion.cursos[i]) {
                        ocupados[asignacion.dia][asignacion.periodo] = true;
                    }

                    break;
                }
            }
        }

        // clases aisladas
        for (int dia = 0; dia < instancia.days; ++dia) {
            for (int periodo = 0; periodo < instancia.periods_per_day; ++periodo) {

                if (!ocupados[dia][periodo]) {
                    continue;
                }

                bool anterior = (periodo > 0) && ocupados[dia][periodo - 1];
                bool siguiente = (periodo < instancia.periods_per_day - 1)
                                 && ocupados[dia][periodo + 1];

                if (!anterior && !siguiente) {
                    penalizacion++;
                }
            }
        }
    }

    return penalizacion;
}
```

```cpp
//S4: Room stability (penalizar por cantidad de salas diferentes para un mismo curso sin contar la primera)
int evaluarS4(const Instancia& instancia, const Solucion& solucion) {
    (void) instancia;
    int penalizacion = 0;

    for (size_t i = 0; i < solucion.cursos.size(); ++i) {

        vector<string> salasUtilizadas;
        //Encotnrar todas las salas diferentes
        for (const Asignacion& asignacion : solucion.cursos[i]) {

            bool encontrada = false;

            for (const string& idSala : salasUtilizadas) {
                if (idSala == asignacion.sala) {
                    encontrada = true;
                    break;
                }
            }

            if (!encontrada) {
                salasUtilizadas.push_back(asignacion.sala);
            }
        }

        // penalización
        if (!salasUtilizadas.empty()) {
            penalizacion += salasUtilizadas.size() - 1; //-1 pq nocuenta la primera
        }
    }

    return penalizacion;
}
```

Caso base

Presentación 1 > codes > ejemplo_base > ejemplo.txt

```
Name: ToyExample
Courses: 4
Rooms: 2
Days: 5
Periods_per_day: 4
Curricula: 2
Constraints: 8

COURSES:
SceCosc Ocra 3 3 30
ArcTec Indaco 3 2 42
TecCos Rosa 5 4 40
Geotec Scarlatti 5 4 18

ROOMS:
A 32
B 50

CURRICULA:
Cur1 3 SceCosc ArcTec TecCos
Cur2 2 TecCos Geotec

UNAVAILABILITY_CONSTRAINTS:
TecCos 2 0
TecCos 2 1
TecCos 3 2
TecCos 3 3
ArcTec 4 0
ArcTec 4 1
ArcTec 4 2
ArcTec 4 3

END.
```

Lectura

```
./lectura


Nombre instancia:  ToyExample

Cabecera:
  - Courses: 4
  - Rooms: 2
  - Days: 5
  - Periods_per_day: 4
  - Curricula: 2
  - Constraints: 8

COURSES:
  - ID: SceCosc | Teacher: Ocra | #Lectures: 3 | Min. days: 3 | #Students: 30
  - ID: ArcTec | Teacher: Indaco | #Lectures: 3 | Min. days: 2 | #Students: 42
  - ID: TecCos | Teacher: Rosa | #Lectures: 5 | Min. days: 4 | #Students: 40
  - ID: Geotec | Teacher: Scarlatti | #Lectures: 5 | Min. days: 4 | #Students: 18

ROOMS:
  - ID: A | Capacity: 32
  - ID: B | Capacity: 50

CURRICULA:
  - ID: Cur1 | Courses: SceCosc ArcTec TecCos
  - ID: Cur2 | Courses: TecCos Geotec

UNAVAILABILITY_CONSTRAINTS:
  - CourseID: TecCos | Day: 2 | Day_period: 0
  - CourseID: TecCos | Day: 2 | Day_period: 1
  - CourseID: TecCos | Day: 3 | Day_period: 2
  - CourseID: TecCos | Day: 3 | Day_period: 3
  - CourseID: ArcTec | Day: 4 | Day_period: 0
  - CourseID: ArcTec | Day: 4 | Day_period: 1
  - CourseID: ArcTec | Day: 4 | Day_period: 2
  - CourseID: ArcTec | Day: 4 | Day_period: 3

nncc@Nico:/mnt/c/Users/nncc2/OneDrive/Documentos/IA Proyecto/Presentación 1/codes$
```

Greedy propuesto

El algoritmo utiliza la lógica de colocar primero aquellas clases que más restricciones tengan. En particular las restricciones de ‘UNAVAILABILITY_CONSTRAINTS’.

1. Elige el curso con mayor restricciones de disponibilidad. En caso de empate elige aquella con mayor cantidad de lectures. En caso de empate nuevamente elige por orden alfabético por CourseID.
2. Posiciona las lectures de las clases en días, periodos y salas que cumplan con las restricciones.
3. Repetir hasta formar la solución inicial.

Si no consigue construir una solución factible, reinicia el algoritmo cambiando el curso inicial por el siguiente curso según el orden de prioridad. Continúa hasta probar cada curso como punto de partida. Si ninguno permite construir una solución inicial factible, retorna solución vacía.

Solución inicial con greedy

```
Solucion encontrada:

Curso 0:
  Dia: 2 | Periodo: 0 | Sala: B
  Dia: 2 | Periodo: 1 | Sala: B
  Dia: 2 | Periodo: 2 | Sala: A

Curso 1:
  Dia: 1 | Periodo: 1 | Sala: A
  Dia: 1 | Periodo: 2 | Sala: A
  Dia: 1 | Periodo: 3 | Sala: A

Curso 2:
  Dia: 0 | Periodo: 0 | Sala: A
  Dia: 0 | Periodo: 1 | Sala: A
  Dia: 0 | Periodo: 2 | Sala: A
  Dia: 0 | Periodo: 3 | Sala: A
  Dia: 1 | Periodo: 0 | Sala: A

Curso 3:
  Dia: 1 | Periodo: 1 | Sala: B
  Dia: 1 | Periodo: 2 | Sala: B
  Dia: 1 | Periodo: 3 | Sala: B
  Dia: 2 | Periodo: 0 | Sala: A
  Dia: 2 | Periodo: 1 | Sala: A

Verificacion de restricciones:
H1: 1
H2: 1
H3: 1
H4: 1


f_eval(S) = 107
Archivo creado en ./OUTPUTS/ejemplo.txt
nncc@Nico:/mnt/c/Users/nncc2/OneDrive/Documentos/Proyecto IA/Presentación 1/codes$
```

Movimiento propuesto

*__Time Move:__ Mueve la lecture al siguiente período disponible.
*__Room Move:__ Mueve la lecture a la siguiente sala disponible.
*__Lecture Move:__ Intercambia con la siguiente lecture en S

Para Hill Climbing + MM la lógica general es avanzar con aquel movimiento que minimice la f_eval. No la primera que la disminuya. Para ello revisamos el vecindario de soluciones completo.

El movimiento propuesto consiste en modificar la asignación de una lecture, probando distintas combinaciones de día, período y sala. Esta propuesta toma como referencia los operadores descritos por [2], que diferencian Time Move*, Room Move* y Lecture Move*.

Luego se ve el vecindario completo para cada operador, se suman todos en un mismo espacio y se elige el mejor.

$$N = \{Time\ Move(S) + Room\ Move(S) + Lecture\ Move(S)\}$$

Este set de movimientos permite suficiente diversificación disminuyendo la posibilidad de movimientos sin mejoras significativas, que sumado a la intensificación de HC, deberían dar buenos resultados.

Trabajo pendiente

- Implementar Hill Climbing

Conclusiones

Bibliografía

1: *Chehade Casivar, N. (2026). Estado del arte: Curriculum-Based Course Timetabling (CB-CTT) [Informe académico].*

2: